Advanced Empirical Finance: Topics and Data Science

Machine learning

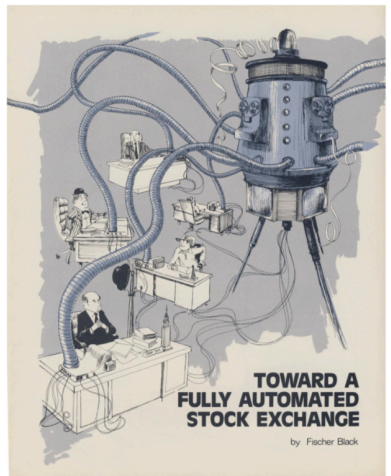
Stefan Voigt

University of Copenhagen and Danish Finance Institute (DFI)

Spring 2026

What is Machine learning?

*The definition of "machine learning" is inchoate and is often context specific. We use the term to describe **(i)** a diverse collection of high-dimensional models for statistical prediction, combined with **(ii)** so-called "regularization" methods for model selection and mitigation of overfit and **(iii)** efficient algorithms for searching among a vast number of potential model specifications. (Gu et al. 2020)*



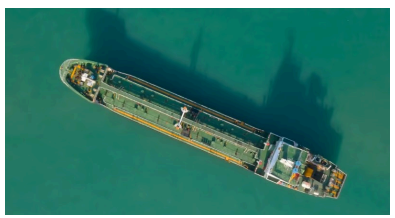
- **(i)** A spectrum from simple linear models to complex architectures, all focused on predictive accuracy
- **(ii)** Selecting among models in-sample leads to overfitting; regularization methods control model complexity
- **(iii)** Searching over vast model spaces creates computational challenges requiring efficient algorithms

What makes ML in Finance special?

Challenges (Israel, Kelly, Moskowitz, 2019)

- Limited data (T limits the left-hand side)
- Markets evolve and thus even lower effective sample size
- By market efficiency: small signal-to-noise ratio (limited predictability)
- Data potentially unstructured (company announcements)





And...

- Machine learning methods on their own do not identify fundamental associations among asset prices and conditioning variables

Overview: Empirical Asset Pricing via Machine Learning

- Familiarize yourself with the paper “Empirical Asset Pricing via Machine Learning” by Gu et al. (2020)
- comparative analysis of machine learning methods for the canonical problem of measuring asset risk premiums
- “We demonstrate large economic gains to investors using *machine learning forecasts*, in some cases doubling the performance of leading regression-based strategies from the literature.”

Machine learning roadmap

1. Bias-Variance Trade-off
2. Penalized Linear Regressions (Ridge and Lasso)
3. Regression Trees and Random Forests
4. Neural Networks
5. Advanced case studies and applications

Your task:

- Return prediction for all CRSP-listed stocks
- Large set of macroeconomic predictors
- Hundreds of predictive firm and economic characteristics
- You should study Gu et al. (2020) in depth!
- **Exercises:** Prepare the dataset as Gu et al. (2020) explain in Section 2.1

ML in Finance: Where are we now?

- Gu et al. (2020) opened the door: large-scale evidence that ML methods outperform linear models for return prediction
- Since then, the field has matured rapidly along several dimensions:

1. **Imposing economic structure improves ML**: No-arbitrage and SDF restrictions often beat unconstrained prediction (Avramov, Cheng & Metzker 2023; Chen, Pelger & Zhu 2024)
2. **Factor models meet ML**: Instrumented PCA (Kelly, Pruitt & Su 2019) and autoencoder-based latent factors (Gu, Kelly & Xiu 2021) bridge classical factor theory and flexible nonlinear models
3. **Characteristic selection**: Nonparametric methods identify which firm characteristics actually matter (Freyberger, Neuhierl & Weber 2020; Bryzgalova, Pelger & Zhu 2024)
4. **External validity**: Out-of-sample replication across markets and time periods remains a challenge (Leippold, Wang & Zhou 2022)
5. **Transaction costs**: Many ML-selected characteristics become irrelevant after accounting for trading frictions (DeMiguel, Martin-Utrera, Nogales & Uppal 2020)

- Key takeaway: The most promising approaches combine statistical flexibility with economic discipline

Bias-Variance Trade-off

Unbiased, linear estimators

$$E_t\left(r_{i,t+1}\right) = g^{\star}(x_{i,t}) \overset{??}{=} \beta' x_{i,t}$$

- Machine learning prescribes a vast collection of high-dimensional models that attempt to predict future quantities of interest while imposing regularization
- We know: OLS is the best linear unbiased estimator (BLUE)
- "Best" = the lowest variance estimator among all other *unbiased linear* estimators
- Requiring the estimator to be *linear* is binding since *nonlinear* estimators exist (e.g., neural networks or regression trees)
- Likewise, *unbiased* is crucial since *biased* estimators do exist

Biased estimators?

- *Shrinkage* methods: the variance of the OLS estimator can be high as OLS coefficients are unregulated
- If we judge by Mean Squared Prediction Error (MSPE), biased estimators could be more attractive if they produce substantially smaller variance than OLS

Shortcomings of OLS

- Let β denote the true regression coefficient and let $\hat{\beta} = (D)^{-1} X'y$, where $D := X'X$, X is a $(T \times N)$ matrix of explanatory variables
- Then, the variance of the (unbiased) OLS estimate $\hat{\beta}$ equals

$$\begin{aligned} Var\left(\hat{\beta}\right) &= E\left(\left(\hat{\beta} - \beta\right)\left(\hat{\beta} - \beta\right)'\right) \\ &= \sigma_{\varepsilon}^2 E\left(D^{-1}\right) \end{aligned}$$

where ε is the vector of residuals and σ_{ε}^2 is the variance of the error term

- When the predictors are highly correlated, the term D^{-1} quickly explodes
- Even worse: the OLS solution is not unique if X is not of full rank

OLS in a prediction context

The Bias-Variance Trade-off

Over-fitting example

- The 40 most recent monthly returns of the manufacturing industry
- Estimate a polynomial regression

$$r_t = \alpha + \sum_{p=1}^{P} \beta_p t^p$$

  where t is a (standardized) time index, on the first 20 months
- Evaluate the performance in-sample (first 20 months) and out-of-sample (last 20 months) for $P = 1, 2, 5, 10, 19$

Ridge Regression

- Hoerl and Kennard (1970a, 1970b) introduced Ridge regression
- Impose a penalty on the L_2 norm of the parameters $\hat{\beta}$ such that for $c \geq 0$ the estimator is

$$\hat{\beta}^{\text{ridge}} = \arg\min_{\beta} (y - X\beta)' (y - X\beta) \text{ s.t. } \beta'\beta \leq c$$

- Standard optimization procedure yields

$$\hat{\beta}^{\text{ridge}} = (D + \lambda I)^{-1} X'y$$

- Hyper parameter λ (c) controls the amount of regularization
- Note that $\hat{\beta}^{\text{ridge}} = \hat{\beta}^{\text{ols}}$ for $\lambda = 0$ $(c \to \infty)$ and $\hat{\beta}^{\text{ridge}} \to 0$ for $\lambda \to \infty$ $(c \to 0)$
- $(D + \lambda I)$ is non-singular even if D is
- *Note:* Usually, we do not penalize the intercept (in practice: demean y)

Ridge Regression: Bias-Variance Properties

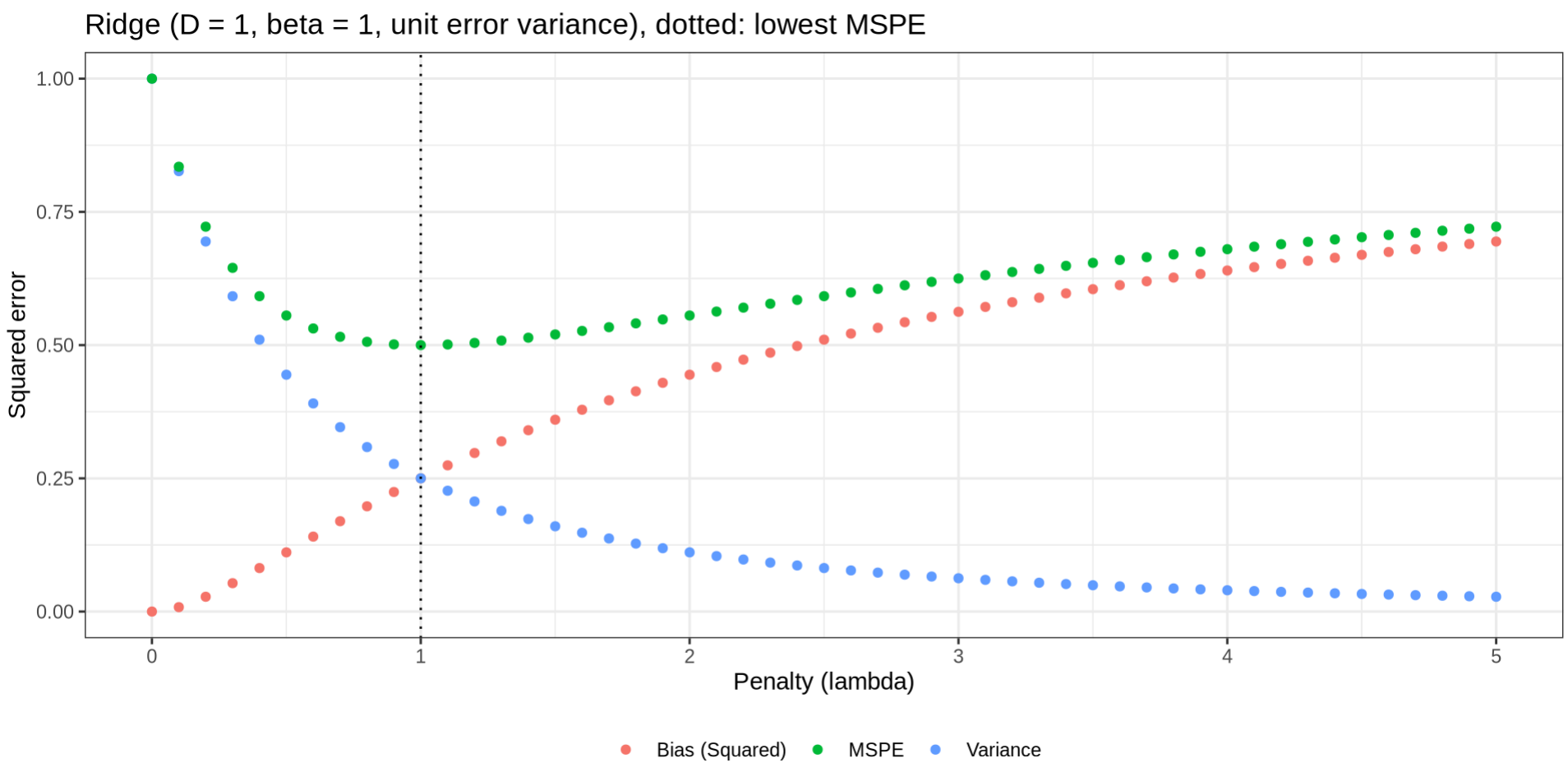
$$
\begin{aligned}
\hat{\beta}^{\text{ridge}} &= (D + \lambda I)^{-1} X'y \\
&= (D + \lambda I)^{-1} DD^{-1} X'y \\
&= \left(D\left(I + \lambda D^{-1}\right)\right)^{-1} D\hat{\beta}^{\text{ols}} \\
&= \left(I + \lambda D^{-1}\right)^{-1} D^{-1} D\hat{\beta}^{\text{ols}} = \left(I + \lambda D^{-1}\right)^{-1} \hat{\beta}^{\text{ols}}
\end{aligned}
$$

- $\hat{\beta}^{\text{ridge}}$ is biased because $E(\hat{\beta}^{\text{ridge}} - \beta) \neq 0$ for $\lambda \neq 0$ - *But* at the same time (under homoscedastic error terms)

$$
\text{Var}(\hat{\beta}^{\text{ridge}}) = \sigma_{\varepsilon}^2 (D + \lambda I)^{-1} X'X (D + \lambda I)^{-1}
$$

- You can show that $\text{Var}(\hat{\beta}^{\text{ridge}}) \leq \text{Var}(\hat{\beta}^{\text{ols}})$ - Trade-off between bias and variance of the estimator!

Bias variance trade-off with the ridge trace



A case study: Explaining industry returns

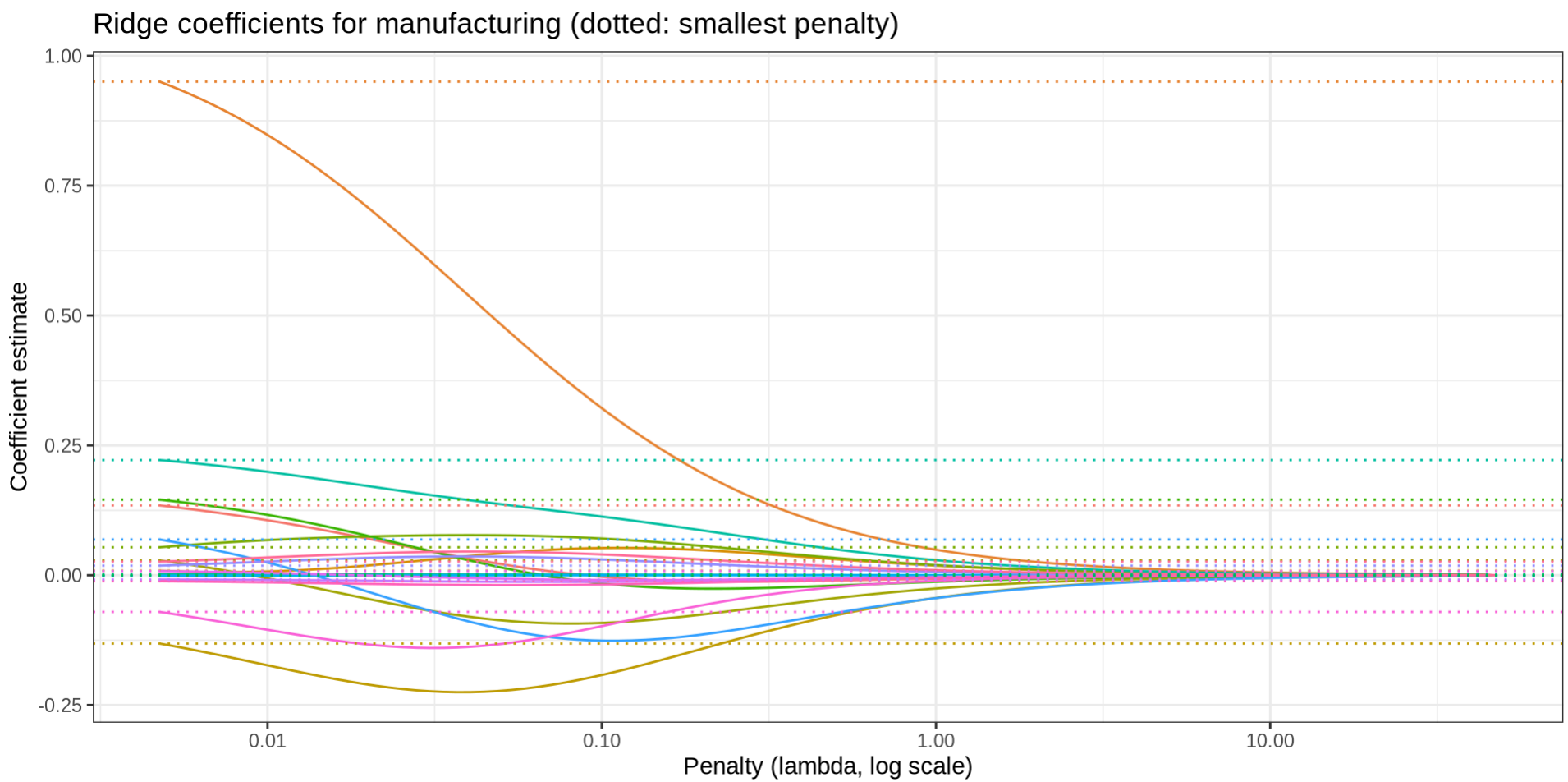
- Data for case study: macroeconomic predictors from the paper "A Comprehensive Look at The Empirical Performance of Equity Premium Prediction" (Welch and Goyal, 2008)
- Monthly variables that the literature has proposed as good predictors for the equity premium: Dividend Price Ratio, Earnings Price Ratio, Stock Variance, Net Equity Expansion, Treasury Bill rate, and inflation
- Monthly Fama-French 3-factor returns (market, small-minus-big, and high-minus-low book-to-market valuation sorts)
- Monthly q-factor returns from Hou, Xue, and Zhang (2015)
- Monthly portfolio returns from 10 different industries according to the definition from Kenneth French's homepage
- The regression specification is

$$\underbrace{r_{i,t}}_{\text{industry } i} = \gamma_{i,0} + \gamma_{i,1}' \underbrace{x_t}_{f_t,\ z_{t-1},\ f_t \times z_{t-1}} + \varepsilon_{i,t}$$

- f_t: factor returns of month t; z_{t-1}: macro predictors of month $t-1$. The model explains industry returns but does not forecast them: f_t is known only at the end of month t
- Package `glmnet` fits generalized linear models via penalized maximum likelihood
- **Exercise:** Implement Ridge on your own before using `glmnet`

Ridge trace

- Below I visualize the *ridge trace* for different values of the penalty λ
- To keep things simple, I restrict the sample to the manufacturing portfolio



The Lasso (Tibshirani, 1996)

- Obvious "drawback" of Ridge regression: no variable selection
- Instead of proportional shrinkage (Ridge), the Lasso (least absolute shrinkage and selection operator) translates each coefficient by a constant factor λ, truncating at zero
- Lasso implements a L_1 penalization on the parameters s.t. $\|\beta\|_1 := \sum_k |\beta_k| \leq c$

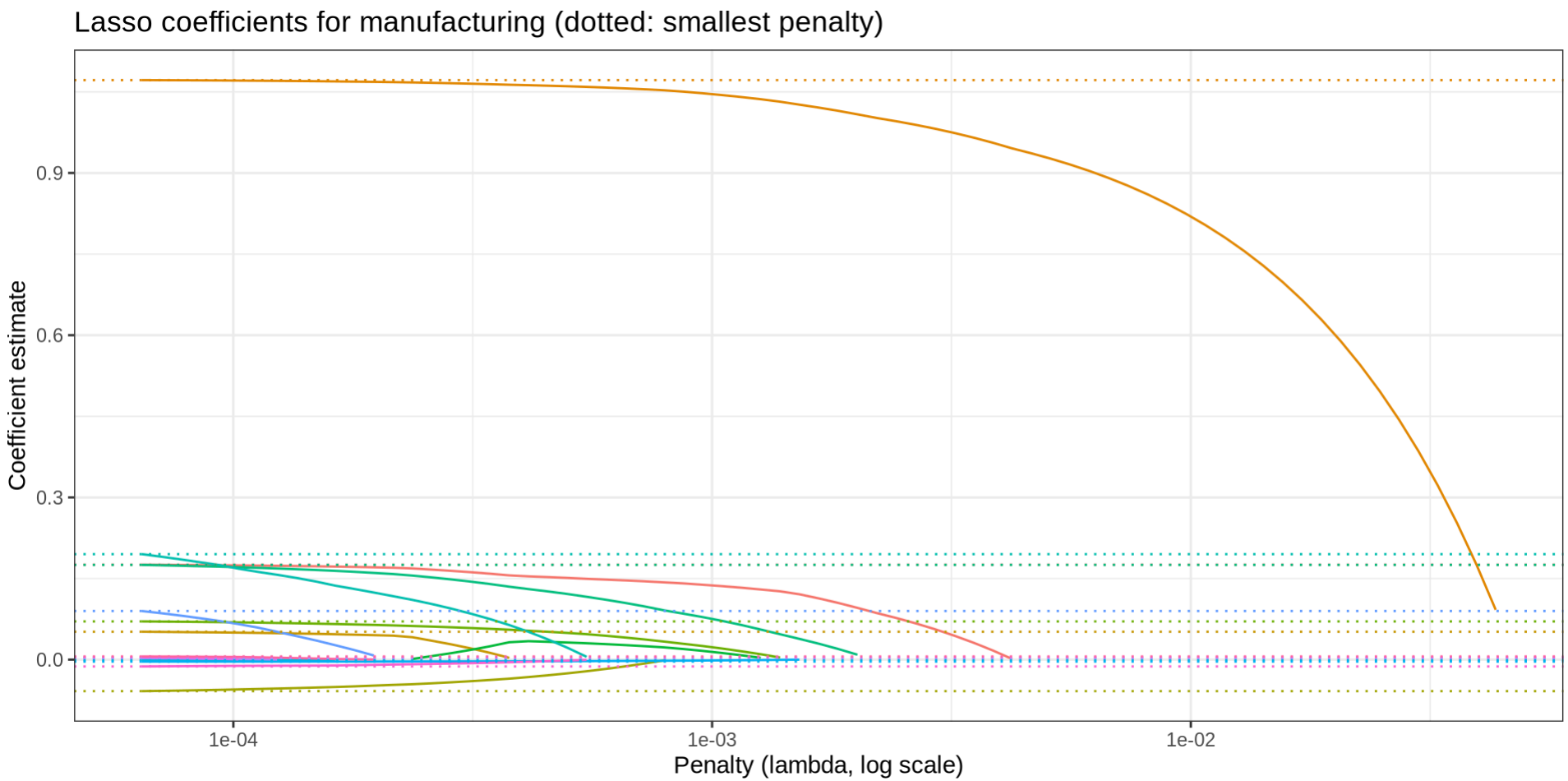
$$\hat{\beta}^{\text{Lasso}} = \arg\min_{\beta} (Y - X\beta)' (Y - X\beta) \text{ s.t. } \sum_{k=1}^{K} |\beta_k| < c(\lambda).$$

- Equivalent optimization problem for a hyperparameter λ:

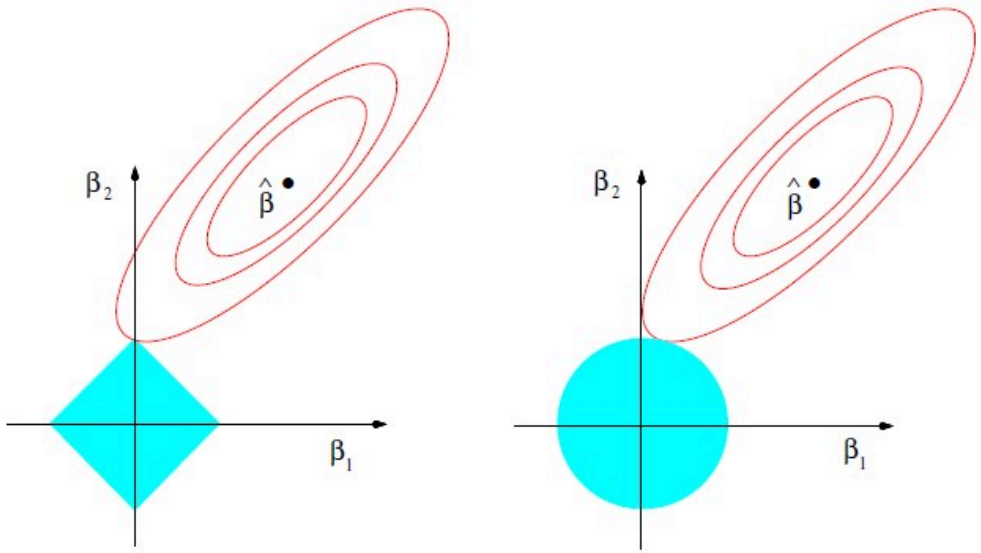
$$\hat{\beta}_{\lambda}^{\text{Lasso}} = \arg\min_{\beta} (Y - X\beta)' (Y - X\beta) + \lambda \sum_{k=1}^{K} |\beta_k|.$$

- No closed-form solution but efficient algorithms (`glmnet`)
- Also here: typically no penalization on the intercept term
- **Exercise:** Implement Lasso estimation on your own before using `glmnet`

Lasso with glmnet



Difference between Ridge and Lasso



- We can interpret Ridge as a Bayesian posterior mean with a **Normal** prior on β
- We can interpret Lasso as a Bayesian posterior mode with a **Laplace** prior on β

$$\beta^{\text{Ridge}} \propto \exp\left(-\frac{\lambda\beta^2}{\sigma}\right) \qquad \beta^{\text{Lasso}} \propto \frac{\lambda}{2\sigma}\exp\left(-\frac{\lambda|\beta|}{\sigma}\right)$$

- Next step: Elastic net (Zou & Hastie, 2005) combines L_1 and L_2 penalization
- Encourages a grouping effect, where strongly correlated predictors tend to be in or out of the model together.

$$\hat{\beta}^{\text{EN}} = \arg\min_{\beta} (Y - X\beta)'(Y - X\beta) + \lambda(1-\rho)\sum_{k=1}^{K}|\beta_k| + \frac{1}{2}\lambda\rho\sum_{k=1}^{K}\beta_k^2$$

Cross-validation

"Hyperparameter-tuning" with cross-validation

- Goal: find an algorithm that produces predictors $\hat{y}$ for an outcome y that minimizes the mean squared prediction error:

$$\text{MSPE} = \text{E}\left(\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2\right)$$

- We can only estimate the MSPE:

$$\hat{\text{MSPE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

1. Because our data is random, the apparent error is a random variable
2. If we train an algorithm on the same dataset that we use to compute the apparent error, we might be overfitting

Cross-validation

- Cross-validation is a technique that permits us to alleviate both these problems
- Think of the true MSPE as the average of many apparent errors that we obtain by applying the algorithm to B new random samples of the data, none of which we use to train the algorithm

$$\frac{1}{B}\sum_{b=1}^{B}\frac{1}{N}\sum_{i=1}^{N}\left(\hat{y}_i^b - y_i^b\right)^2$$

- Idea: randomly generate smaller datasets that we do not use for training and instead use to estimate the true error
- Recall: The goal is to choose hyperparameters λ to obtain the smallest MSPE

Sample split

- Carve out a piece of our dataset and pretend it is an independent dataset: divide it into a *training set* (blue) and a *test set* (red)
- Train the algorithm exclusively on the training set and use the test set only for evaluation purposes (not for filtering out rows, not for selecting features, nothing!)
- Typical choices are to use 10%-20% of the data for testing



Validation sample

- To choose from the set of hyperparameters, we further divide our training sample without using our test sample!
- For each set of algorithm parameters we consider, we want an estimate of the MSPE, and then we will choose the parameters with the smallest MSPE

1. Prespecify a grid of hyperparameters
2. Obtain predictors $\hat{y}_i(\lambda)$ to denote the predictors for the used parameters λ
3. Compute

$$\text{MSPE}(\lambda) = \frac{1}{B}\sum_{b=1}^{B}\frac{1}{N}\sum_{i=1}^{N}\left(\hat{y}_i^b(\lambda) - y_i^b\right)^2$$

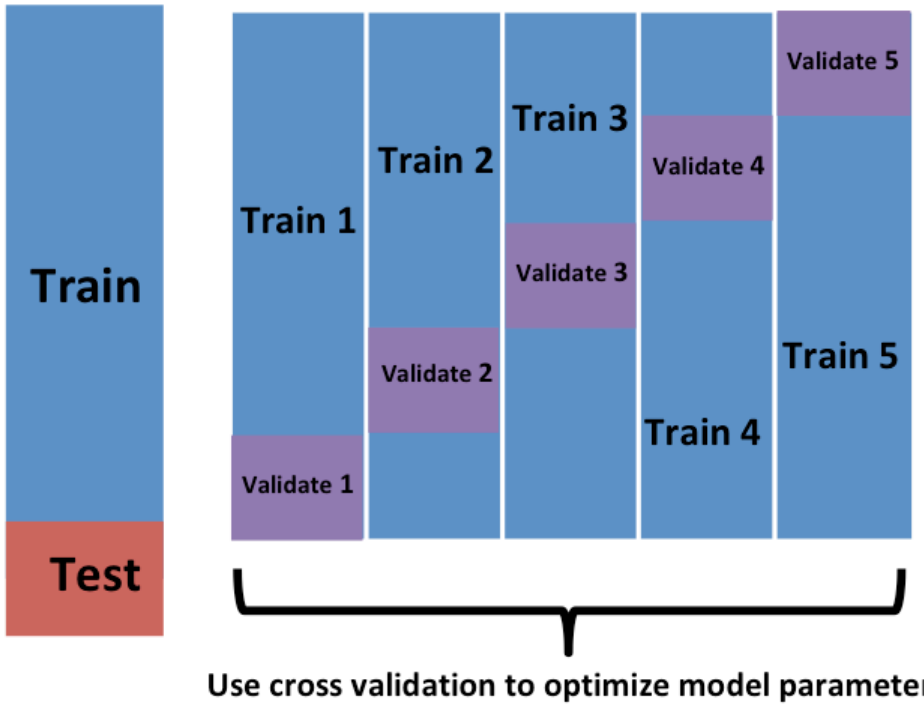
- With K-fold cross-validation, we do it K times: pick a validation set with $M = N/K$ observations at random and think of these as a random sample $y_1^b, \ldots, y_M^b$, with $b = 1$

K-fold cross-validation

- Fit the model in the training set, then compute the apparent error on the independent set

$$\widehat{\mathrm{MSPE}}_b(\lambda) = \frac{1}{M}\sum_{i=1}^{M}\left(\hat{y}_i^b(\lambda) - y_i^b\right)^2$$

- Take K samples to reduce the variance of the estimate
- In K-cross validation, we randomly split the observations into K non-overlapping sets:


Train
Test
Train 1
Validate 1
Train 2
Validate 2
Train 3
Validate 3
Validate 4
Train 4
Validate 5
Train 5
Use cross validation to optimize model parameters

K-fold: Selecting the optimal λ

- Repeat the calculation above for each of these sets $b = 1, \dots, K$ and obtain $\hat{\text{MSPE}}_1(\lambda), \dots, \hat{\text{MSPE}}_K(\lambda)$ and compute the average

$$\hat{\text{MSPE}}(\lambda) = \frac{1}{K} \sum_{b=1}^{K} \hat{\text{MSPE}}_b(\lambda)$$

- final step: select the λ that minimizes the MSPE

Considerations for selecting K

- Large values of K are preferable. The training data better imitates the original dataset
- Larger values of K will have much slower computation time
- One way to improve the variance of our final estimate is to take more samples. To do this, pick K sets of some size at random (not necessarily non-overlapping)
- The bootstrap: at each fold, pick observations at random with replacement (which means the same observation can appear twice)

Cross-validation for time series

- Standard K-fold randomly assigns observations to folds — this creates **look-ahead bias** for time series: the model trains on future data to predict the past
- Financial returns exhibit serial correlation, volatility clustering, and regime changes — random splits destroy this temporal structure

Two alternatives that respect the time ordering:

1. **Expanding window**: training set grows over time, validation is always the next period
    - Pro: uses all available history; Con: trains early and late models on very different sample sizes
2. **Rolling window**: fixed-length training window slides forward
    - Pro: adapts to structural change; Con: discards early data

- Recall from Lecture 2: the same expanding vs. rolling distinction applies to portfolio backtesting
- Additional refinement: **embargo periods** between training and validation sets to avoid information leakage through lagged predictors
- In practice (as in the `tidymodels` example later): `time_series_cv()` and `TimeSeriesSplit()` handle this automatically

Machine learning in one expression

- Method defines function class $\mathcal{F}$ (in this case, linear model) and a regularizer $R(f)$ (shrinkage intensity λ, later: depth of the tree) that expresses the complexity of a function
- Picking the prediction function then involves two steps

1. conditional on a level of complexity, pick the best in-sample loss-minimizing function

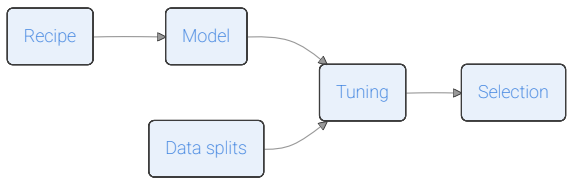
$$\min \sum_{i=1}^{n} L\left(f(x_i), y_i\right) \text{ over } f \in \mathcal{F} \text{ subject to } R(f) \leq c$$

2. estimate the optimal level of complexity c using empirical tuning (cross-validation)

- This regularization framework also applies to portfolio optimization — the mandatory assignment explores how constrained strategies mitigate estimation error

ML workflows in R and Python

- R and Python provide unparalleled workflows for ML: `tidymodels` and `scikit-learn`



- Preprocessing steps (`recipe` and `transformers`)
- Model definition with declaration of tuning parameters (`workflow` and `pipeline`)
- Data split handling
- Tuning and model selection

Tidymodels recipes

```
split <- initial_time_split(
  data |>
    filter(industry == "manuf") |>
    select(-industry),
  prop = 4 / 5
)
```

- We start with a pre-processing plan (`recipe`)
- We remove the column *date*, include all interaction terms between factors and macroeconomic predictors, and demean and scale each variable such that the standard deviation is one
- Understanding why we use recipes instead of `mutate` is key: recipes defer transformations until fitting, which avoids data leakage

```
rec <- recipe(ret_excess ~ ., data = training(split)) |>
  step_rm(date) |>
  step_interact(terms = ~ contains("factor"):contains("macro")) |>
  step_normalize(all_predictors()) |>
  # Demeaned outcome instead of an intercept; skipped for new data (no outcome)
  step_center(ret_excess, skip = TRUE)
```

Scikit-learn recipes

- Scikit-learn offers a similar range of preprocessing (`recipe`) steps
- You have to build interaction terms manually

```
predictors = data.drop("ret_excess", "date", "industry").columns

preprocessor = ColumnTransformer(
    transformers=[("scale", StandardScaler(), predictors)],
    remainder="drop",
    verbose_feature_names_out=False,
)
```

Build a model with tidymodels

- Makes use of a range of packages that `tidymodels` combines

```
lm_model <- linear_reg(penalty = 0.0001, mixture = 1) |>
  set_engine("glmnet", intercept = FALSE)
```

- `lm_model` contains the definition of our model with all required information.
- `set_engine("glmnet")` indicates the API character of the `tidymodels` workflow: Under the hood, the package `glmnet` is doing the heavy lifting, while `linear_reg` provides a unified framework to collect the inputs
- Key advantage: you can swap the model (e.g., change `mixture` to get Ridge, or replace `linear_reg` with a neural network) without changing the rest of the workflow

```
lm_fit <- workflow() |>
  add_recipe(rec) |>
  add_model(lm_model)
```

- `workflow` ends with combining everything necessary for the (serious) data science workflow: a recipe and a model. So now we are ready to use `fit`.

```
lm_fit |> fit(data = training(split))
```

Build a model with Scikit-learn

- Similar to `tidymodels`: Define model and embed in `pipeline`
- `Fit` method to compute parameter estimates

```
lm_model = ElasticNet(
    alpha=0.007,
    l1_ratio=1,
    max_iter=5000,
    fit_intercept=False,
)

lm_pipeline = Pipeline([("preprocessor", preprocessor), ("regressor", lm_model)])

lm_fit = lm_pipeline.fit(
    data_manufacturing_training, data_manufacturing_training["ret_excess"].to_numpy()
)
```

Tune a model with tidymodels

- Recall: Specify a grid of hyperparameters, obtain predictors $\hat{y}_i(\lambda)$ to denote the predictors for the used parameters λ and compute $\text{MSPE}(\lambda)$

```
lm_model <- linear_reg(penalty = tune(), mixture = tune()) |>
  set_engine("glmnet")
lm_fit <- lm_fit |> update_model(lm_model)
```

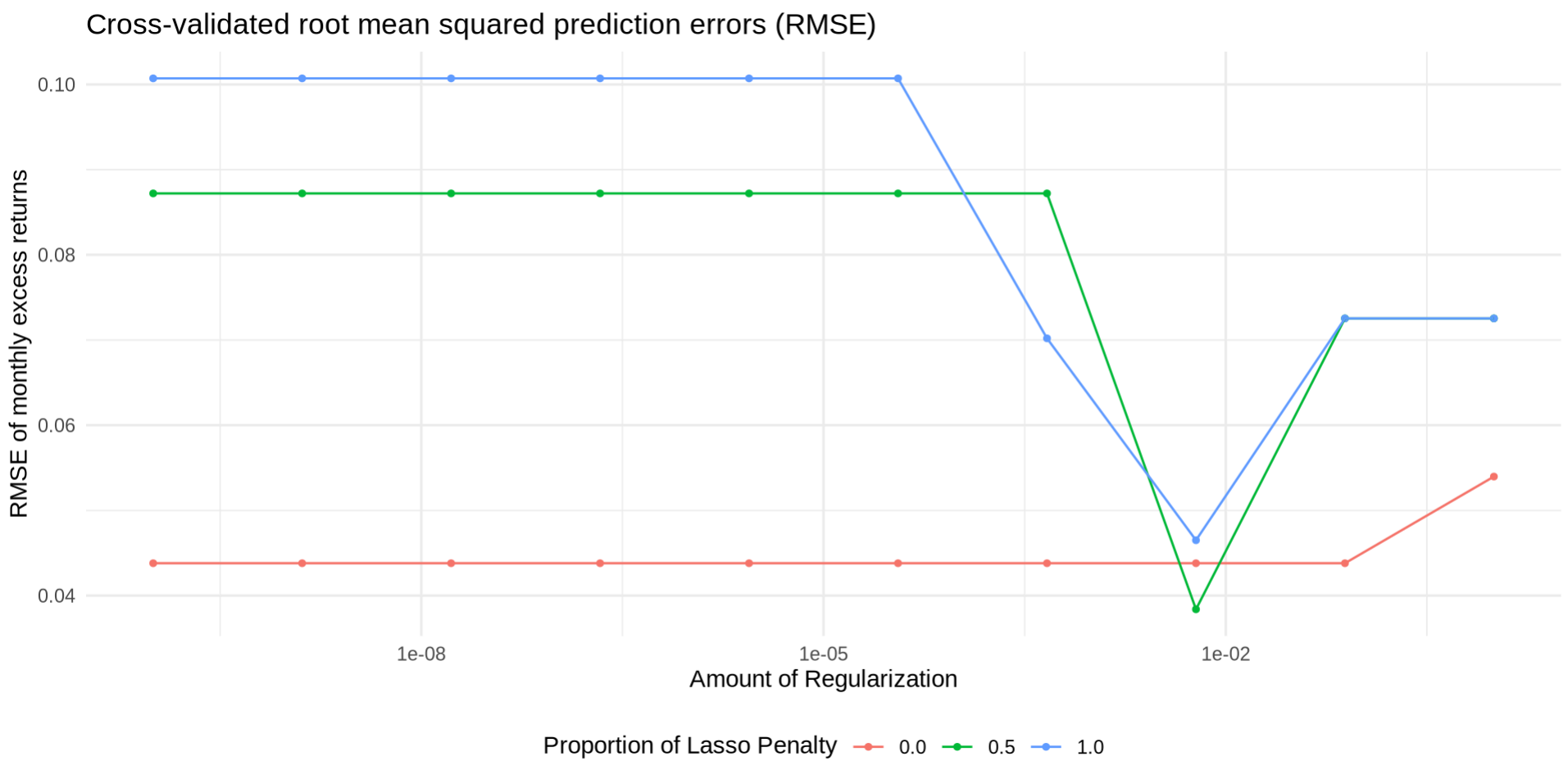
- time-series cross-validation sample: tune with 20 rolling samples of length five years with a validation period of 4 years

```
data_folds <- time_series_cv(
  data = training(split),
  date_var = date,
  initial = "5 years",
  assess = "48 months",
  cumulative = FALSE,
  slice_limit = 20
)
```

```
lm_tune <- lm_fit |>
  tune_grid(
    resamples = data_folds,
    grid = grid_regular(penalty(), mixture(), levels = c(10, 3)),
    metrics = metric_set(rmse)
  )
```

Select the best model

```
autoplot(lm_tune) +
  theme_minimal() +
  theme(legend.position = "bottom") +
  labs(
    title = "Cross-validated root mean squared prediction errors (RMSE)",
    y = "RMSE of monthly excess returns"
  )
```



Tune a model with Scikit-learn

```
initial_months = 60
assessment_months = 48
# TimeSeriesSplit's validation windows do not overlap: as many as fit
n_splits = (len(data_manufacturing_training) - initial_months) // assessment_months

data_folds = TimeSeriesSplit(
    n_splits=n_splits,
    test_size=assessment_months,
    max_train_size=initial_months,
)

params = {
    # The grid of grid_regular(penalty(), mixture(), levels = c(10, 3)) in R
    "regressor__alpha": np.logspace(-10, 0, 10),
    "regressor__l1_ratio": (0.0, 0.5, 1),
}

finder = GridSearchCV(
    lm_pipeline,
    param_grid=params,
    scoring="neg_root_mean_squared_error",
    cv=data_folds,
)

finder = finder.fit(
    data_manufacturing_training, data_manufacturing_training["ret_excess"].to_numpy()
)
```

Advanced Tools

- In the exercises, you will go further: Compute lasso (with penalty as tuning parameter) for **all** industries in the sample
- The figure below illustrates for each industry the selected variables: What are your expectations?

Nonlinear methods

Regression Trees

- Regression trees are fully nonparametric and naturally incorporate multiway predictor interactions
- A tree grows sequentially: at each step, a new branch splits observations into bins based on one predictor variable
- This partitions the predictor space into rectangular regions and approximates $f(x_i)$ with the average outcome within each partition

How do regression trees work?

- We partition the predictor space into J non-overlapping regions, $R_1, R_2, \ldots, R_J$
- For any predictor x that falls within region R_j we estimate $f(x)$ with the average of the training observations y_i for which the associated predictor x_i is also in R_j
- Once we select a partition $\mathbf{x}$ to split to create the new partitions, we find a predictor j and value s that define two new partitions, which we will call $R_1(j, s)$ and $R_2(j, s)$, that split our observations in the current partition by asking if x_j is bigger than s:

$$R_1(j, s) = \{\mathbf{x} \mid x_j < s\} \text{ and } R_2(j, s) = \{\mathbf{x} \mid x_j \geq s\}$$

- To pick j and s, we find the pair that minimizes the residual sum of square (RSS):

$$\sum_{i:\, x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$

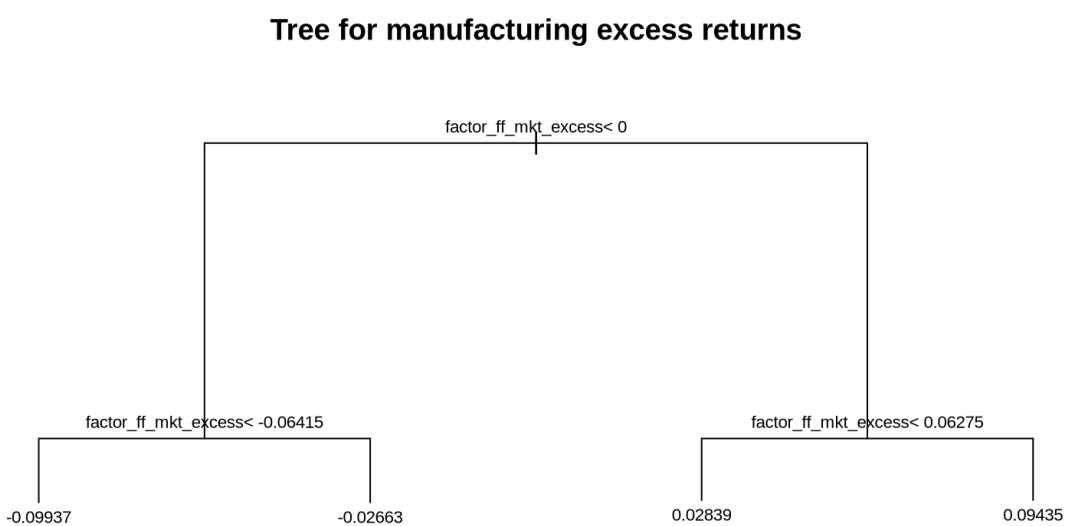
- Note: We do not scale by $\#R_k(j, s)$
- **Discussion:** What are the hyperparameter decisions?

Intuition with R

- Also here: In the **exercises**, you will implement a regression tree on your own

```
# rpart cross-validates the tree internally on randomly drawn folds
set.seed(seed)
tree_model <- rpart(
  ret_excess ~ .,
  data = training(split) |> select(-date),
  control = rpart.control(maxdepth = 2)
)

plot(tree_model, margin = 0.1, main = "Tree for manufacturing excess returns")
text(tree_model, cex = 0.7)
```



Random forests and Bagging

- Single tree models suffer from high variance
- Pruning the tree helps reduce this variance
- Bootstrap aggregating (Bagging, Breiman 1996) averages predictions across multiple trees, reducing variance and overfitting

Growing a forest

- Build B decision trees $T_1, \ldots, T_B$ using the training sample
- The bootstrap: Create a bootstrap training set by sampling N observations from the training set with replacement
- Also: randomly selecting features to include in the building of each tree
- For each observation in the test set, form a prediction $\hat{y} = \frac{1}{B} \sum_{i=1}^{B} \hat{y}_{T_i}$

Hyperparameter tuning for random forests (R)

```r
# Fixed here to keep the example fast; in practice, tune() both
rf_model <- rand_forest(trees = 50, min_n = 20) |>
  set_engine("ranger", seed = seed) |>
  set_mode("regression")
```

- Fitting the model follows the same convention as for the penalized regressions before

```r
rf_fit <- workflow() |>
  add_recipe(rec) |>
  add_model(rf_model) |>
  fit(data = training(split))
```

Hyperparameter tuning for random forests (Python)

```
# Fixed here to keep the example fast; in practice, tune both
rf_model = RandomForestRegressor(
    n_estimators=50,
    min_samples_leaf=20,
    random_state=seed,
)
```

- Fitting the model follows the same convention as for the penalized regressions before

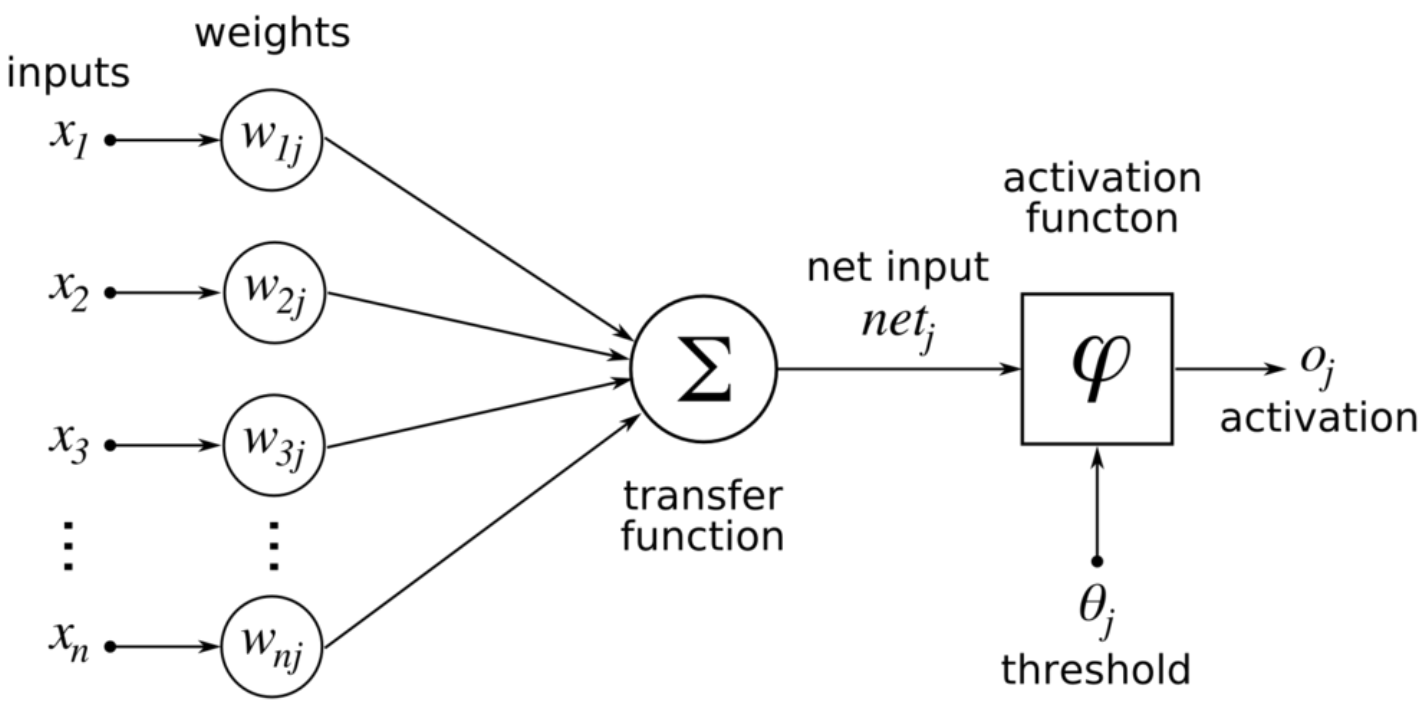
```
rf_pipeline = Pipeline([("preprocessor", preprocessor), ("regressor", rf_model)])

rf_fit = rf_pipeline.fit(
    data_manufacturing_training, data_manufacturing_training["ret_excess"].to_numpy()
)
```

Neural networks

Three major takeaways from the biological neuron

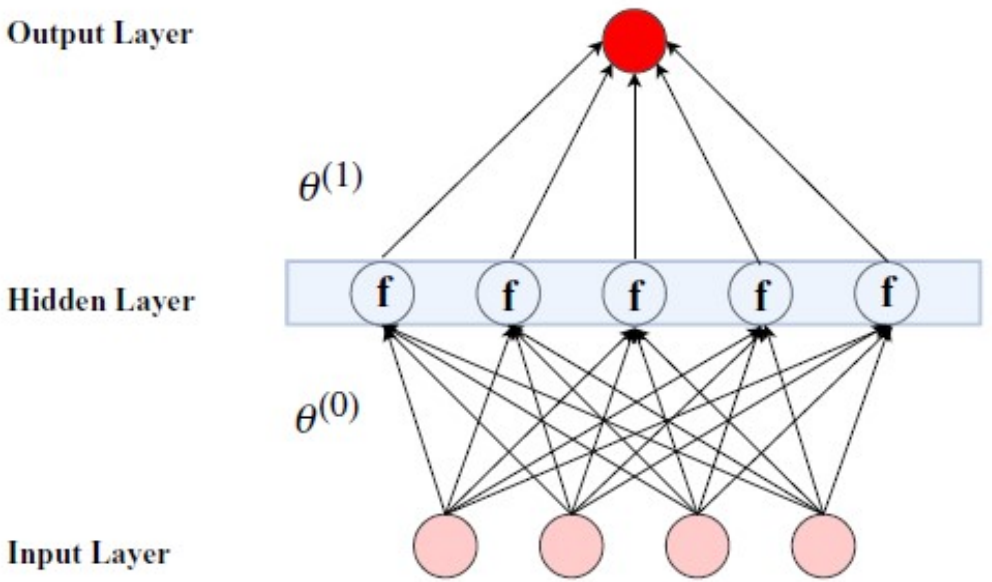
1. The neuron only generates a signal if a sufficient number of input signals enter the neuron's dendrites (all or nothing)
2. Neurons receive inputs from many adjacent neurons upstream and can transmit signals to many adjacent neurons downstream (cumulative inputs)
3. Each neuron has its threshold for activation (synaptic weight)



Feed-forward neural networks

- Neural networks have theoretical underpinnings as “universal approximators” for any smooth predictive association (Hornik, 1991)
- their complexity ranks neural networks among the *least transparent, least interpretable, and most highly parameterized* machine learning tools
- consist of an “input layer” of raw predictors, one or more “hidden layers” that interact and nonlinearly transform the predictors, and an “output layer” that aggregates hidden layers into an outcome prediction
- number of units (neurons) in the input layer is equal to the dimension of the predictors
- output layer usually consists of one neuron (for regression) or multiple neurons for classification

Neural networks: Example



- 4 input units
- One hidden layer with five neurons
- Each neuron receives information from each input unit
- The output layer finally aggregates the results from each neuron, $x_k^1 = f\left(\theta_k^0 + \sum_{j=1}^{4} z_j \theta_{k,j}^0\right)$, into one output forecast

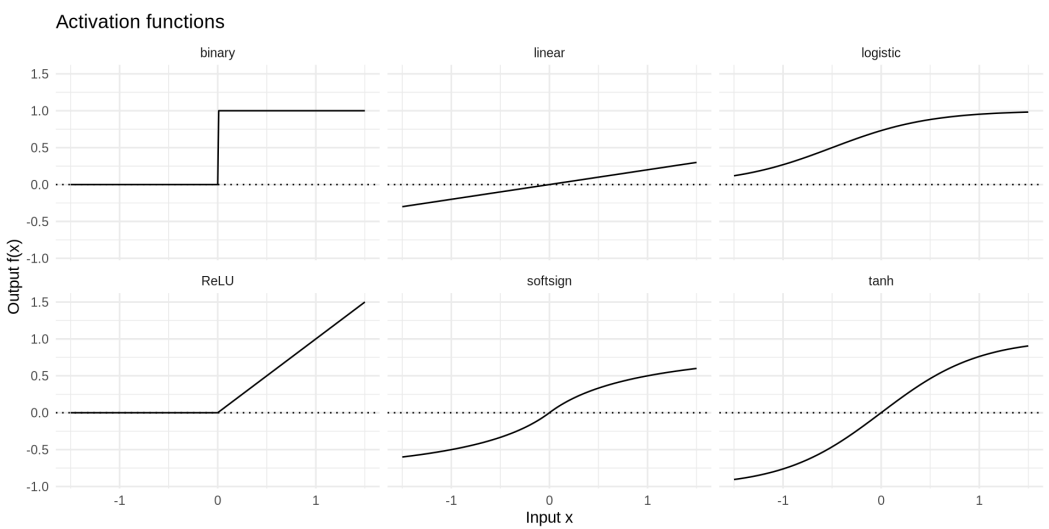
$$\theta_0^1 + \sum_{j=1}^{5} x_j^1 \theta_j^1$$

Activation function

- Each neuron applies a nonlinear "activation function" f to its aggregated signal before sending its output to the next layer

$$x_k^l = f\left(\theta_0^k + \sum_{j=1}^{N^l} z_j \theta_{l,j}^k\right)$$

- Easiest case: $f(x) = \alpha + \beta x$ resembles linear regression
- Typical activation functions are sigmoid ($f(x) = (1 + e^{-x})^{-1}$) or ReLu ($f(x) = max(x, 0)$)



Neural networks: Architecture and Implementation

Plenty of decisions

- Depth (number of hidden layers), Activation function, number of neurons, connections of units (dense or sparse), regularization to avoid overfitting, learning rate
- There is no clear theoretical guidance on these choices but rather a large number of rules of thumb
- Despite the computational challenges, implementation in R and Python is not tedious at all
- Take a look **at this impressive visualization**
- Due to the data transformation process that Neural Networks perform, they are susceptible to the individual scale of the feature values: Standardize the feature sets!

Backpropagation

- How to train the model?
- For given weights $\boldsymbol{w}$, we run the neural network and receive output $\tilde{\boldsymbol{y}}$ as a function of the input values X (as well as the network architecture and the current weights)
- Compute the loss $\mathcal{L}\left(\tilde{y}, y\right)$, e.g. MSPE
- The aim is to choose $\boldsymbol{w}$ to minimize the Loss
- $\Rightarrow$ Differentiate with respect to $\boldsymbol{w}$ and move in the direction of the negative gradient (step size is a hyperparameter, which we often call the learning rate)

$$\Delta\vec{w} = r \cdot \left(\frac{\partial P}{\partial w_0}, \frac{\partial P}{\partial w_1}, ..., \frac{\partial P}{\partial w_q}\right)$$

- For a thorough discussion of backpropagation, consider watching **this lecture by 3Blue1Brown**

Deep neural networks in R

- `torch` provides a helpful interface to create and train a deep neural network
- The example below generates a sequential model with 21 input units, two hidden layers with 64 and 16 neurons, respectively, and ReLu activation functions
- Dropout and L_2 regularization is easy to include
- Output is a single neuron with linear (default) activation function which we can use for predictions

```
library(torch)
library(brulee)

# brulee seeds torch from R's random numbers: call set.seed(seed) before fit()
deep_nnet_model <- mlp(
  epochs = 500,
  dropout = 0.6,
  hidden_units = c(64, 16),
  activation = "relu"
) |>
  set_mode("regression") |>
  set_engine("brulee", verbose = FALSE)
```

Deep neural networks in Python

```
deepnnet_model = MLPRegressor(
    hidden_layer_sizes=(64, 16),
    activation="relu",
    solver="lbfgs",
    max_iter=500,
    random_state=seed,
)

deepnnet_pipeline = Pipeline(
    [("preprocessor", preprocessor), ("regressor", deepnnet_model)]
)
```

Beyond feed-forward networks

- The architectures above are standard feed-forward networks — modern financial ML increasingly uses specialized architectures:

Architecture	Strength	Financial application
Recurrent networks (LSTM, GRU)	Model sequential dependencies	Return prediction with time-varying parameters
Autoencoders	Unsupervised latent factor discovery	Latent asset pricing factors (Gu, Kelly & Xiu 2021)
Convolutional networks (CNN)	Exploit local spatial structure	Price chart pattern recognition (Jiang et al. 2022)
Transformers / Attention	Capture long-range dependencies, process text	Earnings calls, SEC filings, news sentiment
Generative adversarial networks (GAN)	Learn the data-generating process	SDF estimation (Chen, Pelger & Zhu 2024)

- Key trend: architectures that embed economic structure (no-arbitrage, factor models) outperform generic off-the-shelf networks
- Practical implication: start simple (penalized linear models), add complexity only when out-of-sample gains justify it

Empirical Asset Pricing via Machine Learning

- Now you have everything you need to follow Gu et al. (2020): Methods, Data, Procedure

Hyperparameter tuning - Consult the Online Appendix

	OLS-3 +H	PLS	PCR	ENet +H	GLM +H	RF	GBRT +H	NN1 - NN5
Huber loss $\xi =$ 99.9% quantile	✓	-	-	✓	✓	-	✓	-
Others		K	K	$\rho = 0.5$ $\lambda \in (10^{-4}, 10^{-1})$	#Knots=3 $\lambda \in (10^{-4}, 10^{-1})$	Depth= $1 \sim 6$ #Trees= 300 #Features in each split $\in \{3, 5, 10, 20, 30, 50...\}$	Depth= $1 \sim 2$ #Trees= $1 \sim 1000$ Learning Rate LR$\in \{0.01, 0.1\}$	L1 penalty $\lambda_1 \in (10^{-5}, 10^{-3})$ Learning Rate LR$\in \{0.001, 0.01\}$ Batch Size=10000 Epochs=100 Patience=5 Adam Para.=Default Ensemble=10

Note: The table describes the hyperparameters that we tune in each machine learning method.

Results: Portfolio Performance

- Portfolio sorts based on return prediction: at the end of month t, compute one-month-ahead out-of-sample return predictions, sort stocks into deciles, and form a long-short portfolio (decile 10 minus decile 1)

Results: Variable Importance

- Variable importance: for each method, compute the reduction in R^2 from setting all values of a given predictor to zero
- All methods agree on a small set of dominant predictive signals: price trends (reversal and momentum), stock liquidity, stock volatility, and valuation ratios

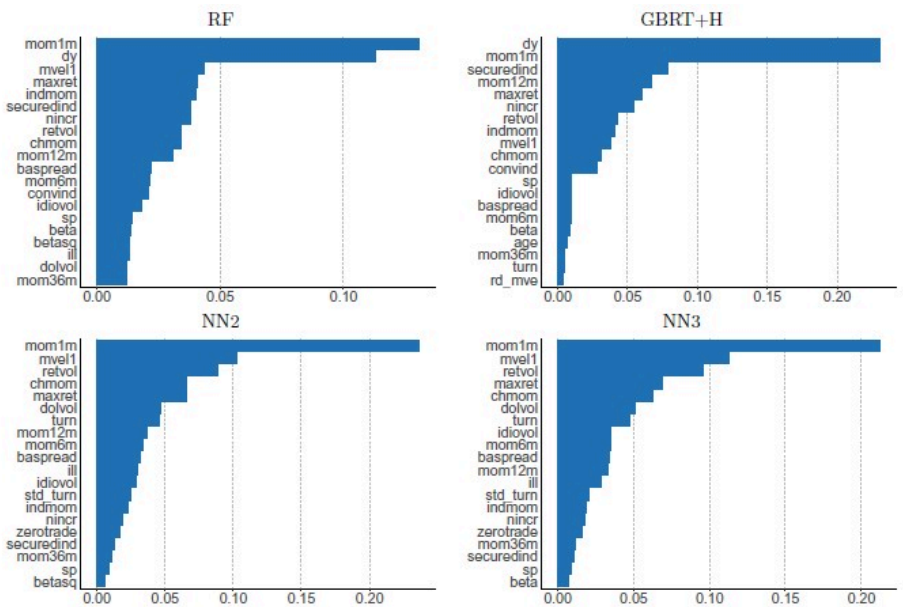


Figure 4
Variable importance by model
Variable importance for the top-20 most influential variables in each model. Variable importance is an average over all training samples. Variable importance within each model is normalized to sum to one.

Interpreting ML models

- Neural networks and random forests are powerful but opaque — how do we understand *what* drives predictions?
- Variable importance (as above) is one approach, but several complementary tools exist:

Method	Idea	Scope
Permutation importance	Shuffle one predictor, measure R^2 drop	Any model
Partial dependence plots	Vary one predictor, average predictions over all others	Any model
Shapley values (SHAP)	Game-theoretic attribution of each predictor's marginal contribution	Any model
Lasso / Elastic Net	Built-in: zero coefficients = excluded predictors	Linear models

- In finance, interpretability is not just academic: regulators and portfolio managers need to understand *why* a model makes a particular prediction
- Partial dependence plots are especially useful for detecting nonlinear predictor effects (e.g., the U-shaped relationship between size and expected returns)

Economic structure vs. pure prediction

- Gu et al. (2020) let the data speak: minimize prediction error without imposing economic theory
- Alternative: embed no-arbitrage restrictions directly into the ML objective

Approach	Example	Key idea
Pure prediction	Gu et al. (2020)	Minimize MSPE for $E(r_{i,t+1})$
SDF estimation	Chen, Pelger & Zhu (2024)	Learn the stochastic discount factor m_{t+1} such that $E(m_{t+1}r_{i,t+1}) = 0$
Latent factors	Kelly, Pruitt & Su (2019); Gu, Kelly & Xiu (2021)	Estimate factor loadings as functions of characteristics

- Counterintuitive finding: imposing economic constraints often *improves* out-of-sample prediction (Avramov, Cheng & Metzker 2023)
- Intuition: constraints act as regularization — reducing the effective parameter space, just like Ridge or Lasso
- This mirrors the lesson from portfolio optimization: constrained strategies outperform unconstrained plug-in estimates when estimation error is large

Characteristic selection and the factor zoo

- Recall from Lecture 3: hundreds of candidate predictors, many likely spurious
- ML offers principled approaches to this selection problem:
  - → **Lasso** and **Elastic Net** perform automatic variable selection (this lecture)
  - → **Adaptive group Lasso** selects characteristics nonparametrically (Freyberger, Neuhierl & Weber 2020)
  - → **Decision-tree portfolios** construct managed portfolios directly from characteristics (Bryzgalova, Pelger & Zhu 2024)
- But: many ML-selected signals lose significance after accounting for transaction costs (DeMiguel, Martin-Utrera, Nogales & Uppal 2020)
- Bottom line: statistical significance $\neq$ economic significance

Out-of-sample replication and overfitting in practice

- Recall from Lecture 3: many published factor results fail to replicate
- ML compounds this problem: hyperparameter tuning introduces additional degrees of freedom for data snooping
- Sources of overfitting specific to ML in finance:
  - → Choice of hyperparameter grid and cross-validation scheme
  - → Selection of predictor sets and sample periods
  - → Architecture decisions (number of layers, neurons, activation functions)
- Leippold, Wang & Zhou (2022) replicate Gu et al. (2020) in the Chinese stock market — results vary substantially across markets and time periods
- What makes the original Gu et al. (2020) results credible?
  - → Comprehensive horse race across many methods
  - → Consistent out-of-sample evaluation period
  - → Economic evaluation (portfolio sorts), not just statistical fit
- Lesson: always evaluate ML models on *economic* criteria (Sharpe ratios, certainty equivalents), not just MSPE

Case studies

Option Pricing

- Recall: The value of a call option for a non-dividend-paying underlying stock in terms of the Black–Scholes parameters is:

$$
\begin{aligned}
C(S_t, t) &= N(d_1)S_t - N(d_2)Ke^{-r(T-t)} \\
d_1 &= \frac{1}{\sigma\sqrt{T-t}}\left[\ln\left(\frac{S_t}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)(T-t)\right] \\
d_2 &= d_1 - \sigma\sqrt{T-t}
\end{aligned}
$$

- C is the price of the option as a function of stock price S and time t, r is the risk-free interest rate, and σ is the volatility of the stock. $N(\cdot)$ is the standard normal cumulative distribution function.
- Can machine learning methods **learn** the Black-Scholes equation after observing different specifications and corresponding prices?

Start with simulated data

- We compute option prices for Call options for a grid of different combinations of maturity (`T`), risk-free rate (`r`), volatility (`sigma`), the strike price (`K`), and current stock price (`S`)
- To make it harder: Add an idiosyncratic error term to each observation

“Learning” Black-Scholes

- We leave the implementation to you
- Models we consider: Deep Neural Network, Lasso (based on a polynomial expansion of all inputs), Random Forests, and a single-layer neural network
- Below: Out-of-sample prediction error for different strike prices

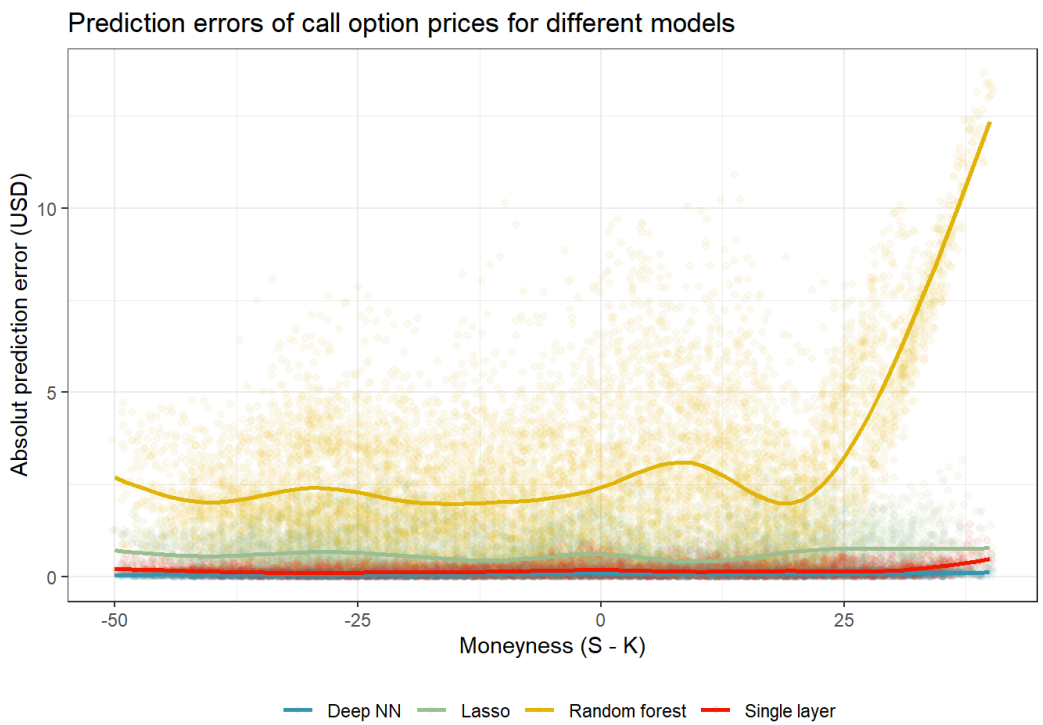
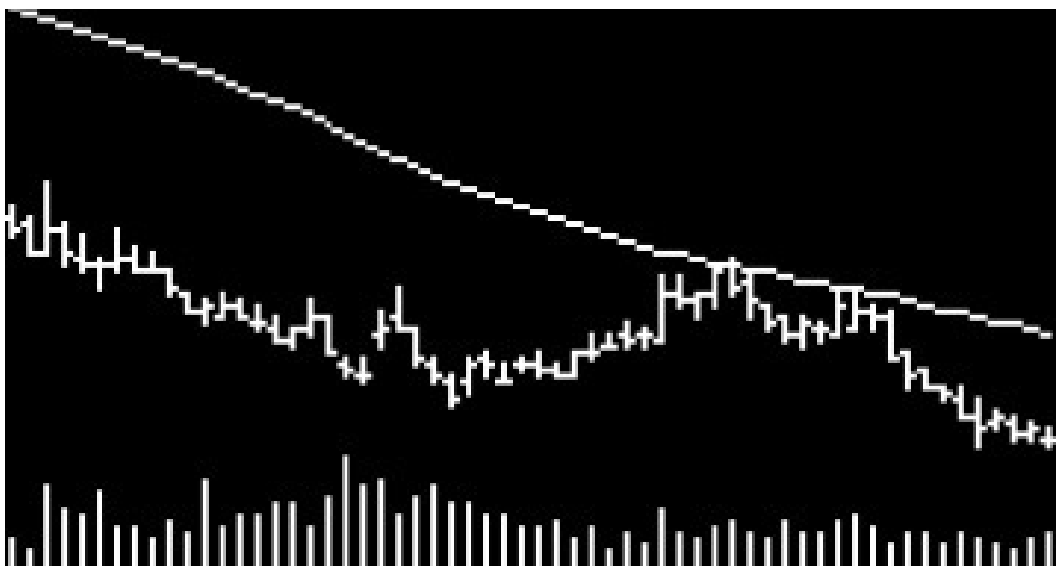


Image recognition

- Jiang et al. (2022) "train" a neural network on financial time series charts
- "reconsider the idea of trend-based predictability using methods that flexibly learn price patterns that are most predictive of future returns, rather than testing hypothesized or pre-specified patterns (e.g., momentum and reversal)."



- We can represent charts as vectors with length *horizon* $\times$ *resolution*
- Convolutional neural network: cross-parameter restrictions that dramatically reduce parameterization relative to standard feed-forward neural networks
- Replication code available on **tidy-finance.org**